

Inorder: 20, 30, 40, 50, 60, 70, 80

Start at 50
↳ go left until null
  ↳ 20 (print) ✓
↳ go back to 30
  ↳ can't go left (print 30) ✓
  ↳ go right to 40
    ↳ 40 going left = null (print 40) ✓
↳ go back to 50
  ↳ can't go left (print 50) ✓
↳ check right side
  ↳ go left until null
    ↳ 60 (print) ✓
  ↳ go back to 70
    ↳ can't go left (print 70) ✓
    ↳ go right to 80
      ↳ 80 going left = null (print 80) ✓
↳ DONE

```
void inorder(Node<T>* node, void (*f)(T&)):
    if (node == nullptr) return nullptr

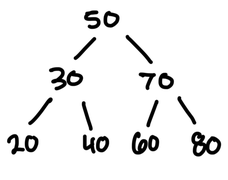
    inorder(node->left, f);
    f(node->data);
    inorder(node->right, f);
```


Preorder: 50, 30, 20, 40, 70, 60, 80

Start at 50 (print) ✓
go left = 30 (print) ✓
go left = 20 (print) ✓
go left = null, go right = null
↳ go back to 30
  ↳ go right = 40 (print) ✓
    ↳ go left = right = null
    ↳ go back to root
check right side
go right = 70 (print) ✓
go left = 60 (print) ✓
↳ go left = right = null
↳ go back to 70
go right = 80 (print) ✓
done

```
void preorder(Node<T>* node, void (*f)(T&)):
    if (node == nullptr) return nullptr;

    f(node->data);
    preorder(node->left, f);
    preorder(node->right, f);
```



Postorder: 20, 40, 30, 60, 80, 70, 50

start at 50 (root)
↳ go left until null, last visited = 20 (print) ✓
↳ go back up = 30, go right = 40
  ↳ can't go left/right, 40 (print) ✓
  ↳ go back up, 30 done (print) ✓
↳ go back to root, check right side
  ↳ go left until null, last visited = 60 (print) ✓
  ↳ go back up = 70, go right = 80
    ↳ can't go left/right, 80 (print) ✓
    ↳ go back up, 70 done (print) ✓
  ↳ back to root, 50 done (print) ✓

```
void postorder(Node<T>* node, void (*f)(T&)):
    if (node == nullptr) return nullptr;

    postorder(node->left, f);
    postorder(node->right, f);
    f(node->data);
```